

TP 08: Simple ANN Tuning Report

Fundamentals of Artificial Intelligence

Manel Mostefaoui

December 9, 2025

1 Introduction

This report presents the results of experiments conducted to tune an Artificial Neural Network (ANN) for customer churn prediction using the Churn_Modelling dataset. The objective was to optimize the ANN architecture and training parameters to improve model performance.

2 Baseline ANN Model

2.1 Architecture

The baseline model consists of the following architecture:

- **Input layer:** 12 features (after preprocessing)
- **Hidden layer 1:** 6 units, activation function = ReLU
- **Hidden layer 2:** 6 units, activation function = ReLU
- **Output layer:** 1 unit, activation function = Sigmoid

2.2 Training Settings

- **Optimizer:** Adam
- **Loss function:** Binary crossentropy
- **Batch size:** 32
- **Epochs:** 100

2.3 Results

- **Test Accuracy:** 86.45%
- **Confusion Matrix:**

$$\begin{bmatrix} 1517 & 78 \\ 193 & 212 \end{bmatrix}$$

Where:

- True Negatives (TN) = 1517: Correctly predicted non-churners
- False Positives (FP) = 78: Non-churners incorrectly predicted as churners
- False Negatives (FN) = 193: Churners incorrectly predicted as non-churners
- True Positives (TP) = 212: Correctly predicted churners

3 Modified Architectures

3.1 Architecture Variants

Three different architecture variants were tested, keeping the training settings constant (Adam optimizer, batch size = 32, epochs = 100).

Model	Architecture	Test Accuracy	Difference
Baseline	2 layers (6, 6) ReLU	86.45%	-
Variant A	1 layer (8) ReLU	85.85%	-0.60%
Variant B	2 layers (16, 8) ReLU + Tanh	85.90%	-0.55%
Variant C	3 layers (12, 12, 6) ReLU	85.75%	-0.70%

Table 1: Comparison of architecture variants

3.2 Confusion Matrices for Variants

Variant A (1 layer, 8 units):

$$\begin{bmatrix} 1494 & 101 \\ 182 & 223 \end{bmatrix}$$

Accuracy: 85.85%

Variant B (2 layers, 16 + 8 units):

$$\begin{bmatrix} 1510 & 85 \\ 197 & 208 \end{bmatrix}$$

Accuracy: 85.90%

Variant C (3 layers, 12 + 12 + 6 units):

$$\begin{bmatrix} 1509 & 86 \\ 199 & 206 \end{bmatrix}$$

Accuracy: 85.75%

3.3 Best Architecture

The **baseline architecture** (2 hidden layers with 6 units each) achieved the highest accuracy of 86.45%. This suggests that:

- The simpler architecture generalizes better on this dataset
- Adding more complexity (additional layers or units) did not improve performance
- The original architecture was already well-suited for this problem

4 Training Settings Variations

Using the best architecture (baseline: 6, 6 units), different training parameters were tested.

Config	Batch Size	Epochs	Optimizer	Accuracy	Diff.
Baseline	32	100	Adam	86.45%	-
Config 1	16	100	Adam	86.40%	-0.05%
Config 2	64	100	Adam	86.15%	-0.30%
Config 3	32	150	Adam	86.05%	-0.40%
Config 4	32	50	Adam	85.95%	-0.50%
Config 5	32	100	SGD (lr=0.01)	85.20%	-1.25%

Table 2: Comparison of training configurations

4.1 Confusion Matrices for Training Variants

Config 1 (Batch size = 16):

$$\begin{bmatrix} 1526 & 69 \\ 203 & 202 \end{bmatrix}$$

Accuracy: 86.40%

Config 2 (Batch size = 64):

$$\begin{bmatrix} 1508 & 87 \\ 190 & 215 \end{bmatrix}$$

Accuracy: 86.15%

Config 3 (Epochs = 150):

$$\begin{bmatrix} 1520 & 75 \\ 204 & 201 \end{bmatrix}$$

Accuracy: 86.05%

Config 4 (Epochs = 50):

$$\begin{bmatrix} 1516 & 79 \\ 202 & 203 \end{bmatrix}$$

Accuracy: 85.95%

Config 5 (SGD optimizer):

$$\begin{bmatrix} 1521 & 74 \\ 222 & 183 \end{bmatrix}$$

Accuracy: 85.20%

4.2 Best Training Configuration

The **baseline configuration** (batch size = 32, epochs = 100, Adam optimizer) achieved the best results. Key findings:

- Smaller batch size (16) performed nearly as well as baseline
- Larger batch size (64) slightly reduced performance
- Training for more epochs (150) did not improve results, suggesting possible overfitting
- Training for fewer epochs (50) led to underfitting
- SGD optimizer performed significantly worse than Adam

5 Comparison with SVM and Random Forest

Model	Test Accuracy
Best ANN (Baseline)	86.45%
SVM (RBF kernel)	86.50%
Random Forest (100 trees)	86.60%

Table 3: Comparison of different machine learning models

5.1 Confusion Matrices

SVM (RBF kernel):

$$\begin{bmatrix} 1550 & 45 \\ 225 & 180 \end{bmatrix}$$

Accuracy: 86.50%

Random Forest (100 trees):

$$\begin{bmatrix} 1522 & 73 \\ 195 & 210 \end{bmatrix}$$

Accuracy: 86.60%

5.2 Analysis

All three models achieved very similar performance:

- **Random Forest** achieved the highest accuracy (86.60%)
- **SVM** performed slightly better than ANN (86.50% vs 86.45%)
- **ANN** achieved competitive performance with minimal tuning
- The differences are marginal (within 0.15%), suggesting all models are well-suited for this task
- Random Forest may be preferred due to its interpretability and robustness

6 Key Learnings

Through these experiments, several important insights about ANN design and training were gained:

1. **Model Complexity:** More complex architectures (more layers or units) do not always lead to better performance. The simpler baseline architecture generalized better, demonstrating that model complexity should match the problem's inherent complexity.
2. **Batch Size Effects:** Batch size significantly affects training dynamics. Smaller batches (16) provide more frequent weight updates with higher variance, while larger batches (64) offer more stable but less frequent updates. The medium batch size (32) provided the best balance.
3. **Training Duration:** More epochs do not guarantee better results. Training beyond 100 epochs led to slight performance degradation, indicating possible overfitting. Conversely, 50 epochs were insufficient for the model to fully learn the patterns.
4. **Optimizer Selection:** The choice of optimizer is crucial. Adam significantly outperformed SGD (86.45% vs 85.20%), demonstrating the importance of adaptive learning rate methods for this type of problem.
5. **Activation Functions:** Different activation functions (ReLU vs Tanh) can affect convergence and final performance. ReLU consistently performed well across different architectures.
6. **Model Comparison:** Traditional machine learning methods (SVM and Random Forest) can match or slightly exceed ANN performance on structured tabular data, especially when datasets are moderately sized. ANNs may require more sophisticated architectures or larger datasets to show clear advantages.
7. **Hyperparameter Sensitivity:** Small changes in hyperparameters can lead to noticeable performance variations, emphasizing the importance of systematic hyperparameter tuning.

7 Conclusion

This experiment demonstrated that careful tuning of ANN architecture and training parameters is essential for optimal performance. The baseline model with 2 hidden layers (6 units each), Adam optimizer, batch size of 32, and 100 epochs achieved the best ANN performance at 86.45% accuracy. However, Random Forest slightly outperformed all other models at 86.60%, suggesting that for this particular dataset and problem, ensemble methods may be more effective than neural networks.

The marginal differences between models (all within 0.15%) indicate that the choice of algorithm should also consider factors such as training time, interpretability, and deployment requirements, not just raw accuracy.